LAPORAN PRAKTIKUM PEKAN 5

PEMOGRAMAN BERORIENTASI OBJEK

Transaksi Dinamis (Polymorphism)



Oleh:
Naysha Aprilia Rizki
2511531014
Kelas B

Dosen Pengampu:
Nurfiah,S.ST., M.Kom.

Asisten Praktikum:
Ezza Addini

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan sistem perbankan, setiap jenis rekening dapat memiliki aturan transaksi yang berbeda. Contohnya, rekening tabungan memiliki aturan penarikan berdasarkan saldo yang tersedia, sedangkan rekening giro memungkinkan penarikan melebihi saldo hingga batas overdraft tertentu. Jika setiap jenis rekening memiliki implementasi transaksi yang berbeda, diperlukan cara agar program dapat menjalankan aturan tersebut secara fleksibel.

Salah satu konsep Pemrograman Berorientasi Objek yang dapat digunakan adalah *Polymorphism* atau polimorfisme. Konsep ini memungkinkan method dengan nama yang sama memiliki perilaku berbeda sesuai dengan jenis objeknya. Dalam praktikum ini, polimorfisme diterapkan melalui *Method Overriding* pada method tarikTunai() di kelas RekeningGiro, serta *Dynamic Method Dispatch* untuk menentukan method yang dijalankan saat program berlangsung. Selain itu, konsep ini digunakan dalam pengembangan fitur transfer antar rekening.

1.2 Tujuan

1. Memahami konsep polimorfisme dalam Pemrograman Berorientasi Objek.
2. Memahami penerapan *Method Overriding* pada kelas turunan.
3. Memahami konsep *Dynamic Method Dispatch* dalam Java.
4. Menerapkan anotasi `@Override` pada method yang ditimpa.
5. Mengembangkan fitur transaksi yang dapat menyesuaikan jenis rekening.
6. Menerapkan polimorfisme dalam pembuatan fitur transfer antar rekening.

1.3 Manfaat

1. Memahami penerapan polimorfisme dalam pemrograman Java.
2. Mengurangi kebutuhan penggunaan percabangan untuk menangani perilaku objek yang berbeda.
3. Mempermudah pengembangan program dengan aturan transaksi yang beragam.
4. Meningkatkan pemahaman tentang penggunaan *Method Overriding* dan *Dynamic Method Dispatch*.
5. Melatih kemampuan dalam mengembangkan fitur transaksi perbankan menggunakan konsep Pemrograman Berorientasi Objek.

BAB II

PEMBAHASAN

2.1 Teori

Polimorfisme merupakan salah satu konsep dalam Pemrograman Berorientasi Objek yang memungkinkan satu method memiliki perilaku berbeda sesuai dengan jenis objek yang menjalankannya. Dalam sistem perbankan, konsep ini dapat diterapkan pada proses penarikan tunai, ketika rekening biasa dan rekening giro memiliki aturan penarikan yang berbeda.

1. **Polimorfisme (*Polymorphism*)**

Polimorfisme berarti memiliki banyak bentuk. Dalam Java, konsep ini memungkinkan pemanggilan method yang sama menghasilkan perilaku berbeda berdasarkan jenis objek sebenarnya saat program dijalankan. Pada sistem perbankan, method tarikTunai() dapat memiliki aturan berbeda untuk rekening biasa dan rekening giro.

2. **Method Overriding**

Method Overriding adalah proses ketika kelas turunan mendefinisikan kembali method yang berasal dari kelas induk dengan nama dan parameter yang sesuai. Dalam praktikum ini, kelas RekeningGiro menimpa method tarikTunai() agar penarikan dapat dilakukan hingga batas overdraft yang ditentukan.

3. **Dynamic Method Dispatch**

Dynamic Method Dispatch merupakan mekanisme Java dalam menentukan method yang akan dijalankan berdasarkan jenis objek sebenarnya saat program berjalan. Dengan mekanisme ini, pemanggilan akunAktif.tarikTunai() dapat menjalankan

implementasi milik Rekening atau RekeningGiro sesuai objek yang digunakan, tanpa perlu membuat percabangan khusus untuk setiap jenis rekening.

4. **Anotasi @Override**

Anotasi @Override digunakan untuk menandai bahwa suatu method pada kelas turunan dimaksudkan untuk menimpa method dari kelas induk. Anotasi ini membantu compiler mendeteksi kesalahan, seperti ketidaksesuaian nama atau parameter method.

5. **Penerapan Polimorfisme pada Fitur Transfer**

Polimorfisme juga dapat diterapkan pada fitur transfer antar rekening melalui method transfer(Rekening tujuan, double nominal, String pinInput). Parameter Rekening tujuan memungkinkan method menerima objek dari berbagai kelas turunan rekening. Proses transfer dilakukan melalui autentikasi PIN, penarikan dana dari rekening pengirim, dan penyetoran dana ke rekening tujuan apabila penarikan berhasil.

2.2 Langkah Langkah

Praktikum 5 ini, melanjutkan yang sudah dibuat dari praktikum 1 sampai praktikum 4, hanya menambahkan beberapa bagian di beberapa class saja. Class yang digunakan tetap sama seperti Praktikum sebelumnya yaitu Class Transaksi, Class Rekening, Class Rekening Tabungan, Class Rekening Giro, Class Main. Dan yang akan diubah yaitu class yang akan saya jelaskan:

1. **Membuat Class Rekening**

   Salin Code Program yang sudah dibuat pada praktikum prak4_Inheritance, Dan ubah dibagian tarikTunai saja. Berikut Screenshot

```
public boolean tarikTunai(double nominal) {
    if (nominal <= 0) {
        System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
        return false;
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi, Saldo Anda: Rp" + saldo);
            return false;
        } else {
            saldo -= nominal;

    String idTrx = "TRX-T-" + System.currentTimeMillis();
    riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

    System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
    return true; //transaksi sukses
        }
}
```

Penjelasan:

```
public boolean tarikTunai(double nominal) {
```

Diubah dari public void tarikTunai(double nominal) menjadi public boolean tarikTunai(double nominal) karena pada Praktikum 5, method tarikTunai() harus mengembalikan status berhasil(true) atau gagalnya(false) transaksi untuk mendukung fitur transfer antar rekening.

```
if (nominal <= 0) {
    System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
    return false;
```

Jika nominal kecil sama 0 maka Transaksi gagal dan akan mengeluarkan output “Gagal: Nominal tarik harus lebih dari 0!”, kemudian mengembalikan nilai false sebagai tanda bahwa transaksi gagal.

```
} else if (nominal > saldo) {
    System.out.println("Transaksi Gagal: Saldo tidak mencukupi, Saldo Anda: Rp" + saldo);
    return false;
```

Selain itu, Jika nominal yang diinputkan lebih dari saldo maka transaksi akan gagal, dan akan mengeluarkan output “Transaksi Gagal: Saldo tidak mencukupi, Saldo Anda: Rp” + jumlah saldo yang berada pada rekening, kemudian mengembalikan nilai false sebagai tanda bahwa transaksi gagal.

```
        } else {
            saldo -= nominal;

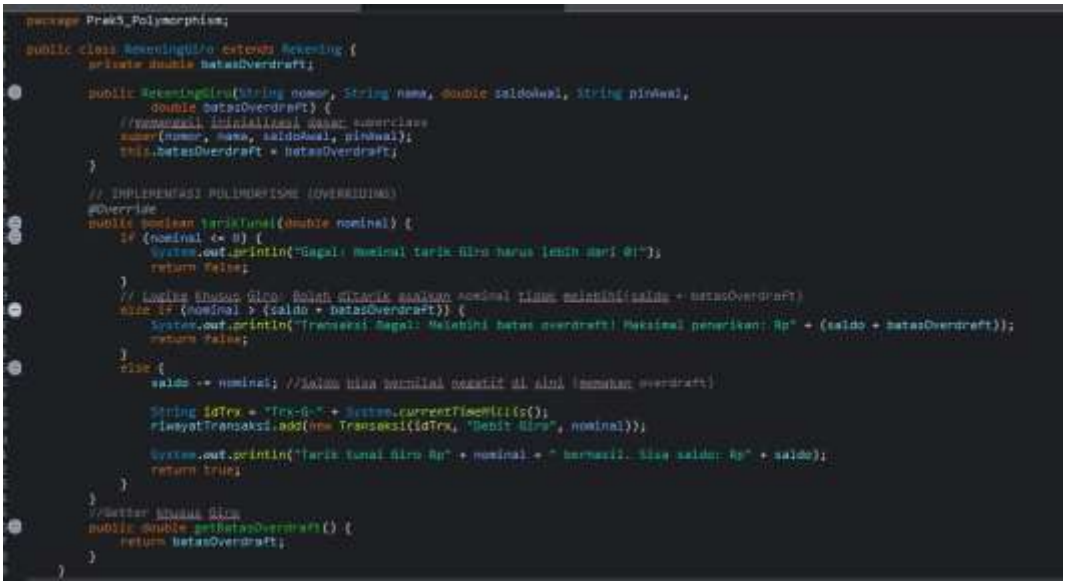
    String idTrx = "TRX-T-" + System.currentTimeMillis();
    riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

    System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
    return true; //transaksi sukses
        }
}
```

Jika nominal penarikan tidak melebihi saldo rekening dan lebih dari 0, maka transaksi akan berhasil. Saldo rekening akan dikurangi sesuai

nominal yang ditarik. Selanjutnya, sistem akan membuat ID transaksi dengan awalan “TRX-T-” berdasarkan waktu saat ini dan menyimpan data transaksi ke dalam riwayat dengan jenis “Debit”. Setelah itu, program akan menampilkan pesan bahwa penarikan berhasil beserta sisa saldo rekening, kemudian mengembalikan nilai true sebagai tanda bahwa transaksi berhasil.

2. Class RekeningGiro



Penjelasan

```
public class RekeningGiro extends Rekening {
```

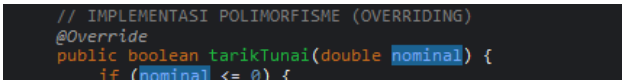
Membuat RekeningGiro sebagai subclass dari Rekening, sehingga mewarisi atribut dan method dari class induk.

```
private double batasOverdraft;
```

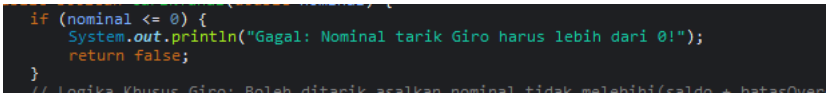
Menyimpan batas overdraft, yaitu batas saldo negatif yang dapat digunakan pada rekening giro.



method tarikTunai() pada class RekeningGiro digunakan untuk melakukan penarikan uang dengan aturan khusus, yaitu pengguna dapat menarik uang melebihi saldo yang tersedia selama jumlah penarikan tidak melampaui batas overdraft. Method ini juga memvalidasi nominal penarikan, mengurangi saldo, mencatat transaksi ke dalam riwayat, menampilkan hasil transaksi, serta mengembalikan nilai true jika berhasil dan false jika gagal. Penerapan @Override menunjukkan konsep polimorfisme, sehingga rekening giro memiliki aturan penarikan yang berbeda dari rekening biasa. Penjelasannya:



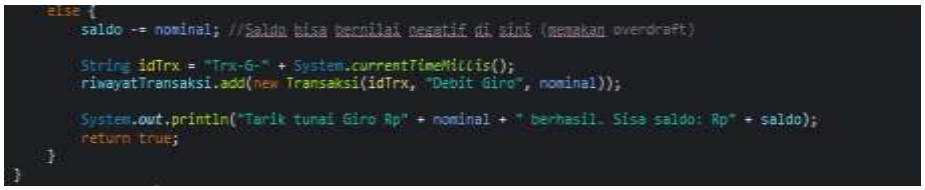
@Override digunakan untuk menandakan bahwa method tarikTunai() pada class RekeningGiro menimpa method milik class induk Rekening. Method ini menggunakan tipe boolean agar dapat mengembalikan nilai true jika transaksi berhasil dan false jika gagal.



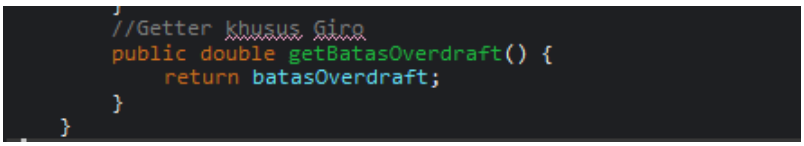
Jika nominal yang diinputkan kurang dari atau sama dengan 0, maka transaksi akan gagal dan program menampilkan pesan bahwa nominal penarikan harus lebih dari 0. Nilai false dikembalikan untuk menandakan bahwa transaksi tidak berhasil.



Jika nominal penarikan melebihi jumlah saldo ditambah batas overdraft, maka transaksi akan gagal. Program menampilkan pesan bahwa penarikan melebihi batas overdraft beserta jumlah maksimal penarikan yang diperbolehkan, kemudian mengembalikan nilai false.

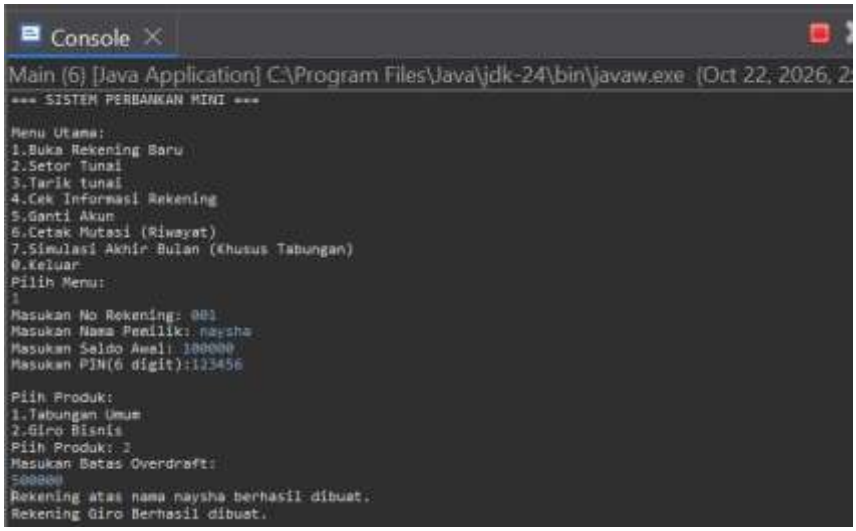


Jika nominal penarikan memenuhi ketentuan batas overdraft, maka saldo rekening akan dikurangi sesuai nominal yang ditarik. Selanjutnya, sistem akan membuat ID transaksi dengan awalan "Trx-G-" berdasarkan waktu saat ini dan menyimpan data transaksi ke dalam riwayat dengan jenis "Debit Giro". Setelah itu, program menampilkan pesan bahwa penarikan berhasil beserta sisa saldo rekening, kemudian mengembalikan nilai true sebagai tanda bahwa transaksi berhasil.



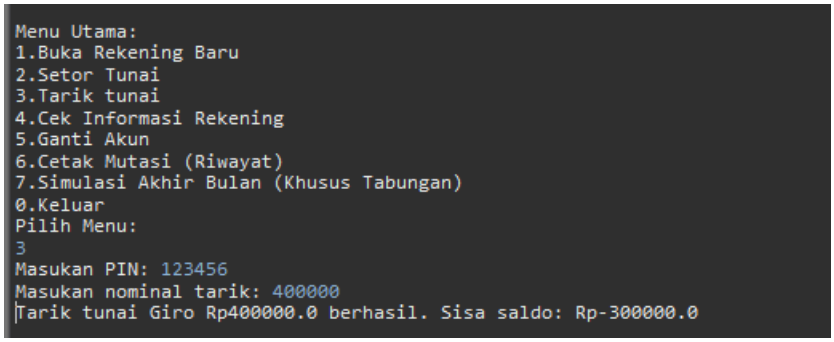
Merupakan getter yang digunakan untuk mengambil nilai batas overdraft karena atribut batasOverdraft bersifat private.

OUTPUT:



Pilih 1, untuk membuka rekening baru dan masukan saldo awal sebesar Rp 100.000.

Pilih Produk 2, yaitu Giro Bisnis masukan batas Overdraftnya Rp 500.000



Pilih menu 3, yaitu Tarik Tunai. Masukan nominal nya Rp 400.000 Karna transaksi ini di Giro tetap bisa dilakukan tapi Sisa saldo jadi Rp -300.000.

LATIHAN

1. Buat method Transfer di class Rekening, tambahkan satu method bar dengan nama : public void transfer(Rekening tujuan, double nominal, String pinInput)

• Method ini menerima parameter polimorfik Rekening tujuan, bisa menerima Tabungan atau Giro.

• Logika Method

▪ Pertama, lakukan otentikasi(pinInput). Jika false, batalkan.

▪ Kedua, panggil this.tarikTunai(nominal). (overriding, jika this adalah Giro, ia otomatis menggunakan aturan overdraft).

▪ Ketiga, periksa kembalian nilai tarikTunai. Jika true, panggil tujuan.setorTunai(nominal).

2. Integrasi ke CLI Utama: Buka Main.java dan tambahkan menu 8. Transfer ke Rekening Lain.

• Minta pengguna memasukkan Nomor Rekening Tujuan.

• Cari nomor tersebut di dalam daftarRekening.

• Jika ketemu, minta nominal transfer dan PIN, lalu panggil akunAktif.transfer(akunTujuan, nominal, pin).

Jawab:

1. Method transfer di class Rekening

```
public void transfer(Rekening tujuan, double nominal, String pinInput) {
    if (!otentikasi(pinInput)) {
        System.out.println("PIN salah!");
        return;
    }

    if (this.tarikTunai(nominal)) {
        tujuan.setorTunai(nominal);
        System.out.println("Transfer berhasil!");
    }
}
```

Penjelasan:

```
public void transfer(Rekening tujuan, double nominal, String pinInput) {
    if (!otentikasi(pinInput)) {
        System.out.println("PIN salah!");
        return;
    }
```

Jika PIN yang dimasukkan tidak sesuai, maka program akan menampilkan pesan “PIN salah!” dan menghentikan proses transfer menggunakan return, sehingga penarikan dan penyetoran ke rekening tujuan tidak dilakukan.

```
    if (this.tarikTunai(nominal)) {
        tujuan.setorTunai(nominal);
        System.out.println("Transfer berhasil!");
    }
}
```

Jika penarikan uang berhasil dan method tarikTunai(nominal) mengembalikan nilai true, maka program akan menyetorkan uang ke rekening tujuan melalui method setorTunai(nominal) dan menampilkan pesan “Transfer berhasil!”. Jika penarikan gagal, maka penyetoran tidak dilakukan dan pesan keberhasilan tidak ditampilkan.

2. Menu Main

1) Tambahkan Menu 8. Transfer ke Rekening Lain

```
System.out.println("8. Transfer ke Rekening Lain");
```

2) Case 8

```
case 8:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan Nomor Rekening Tujuan: ");
        String nomorTujuan = input.nextLine();

        Rekening akunTujuan = null;

        for (Rekening rekening : daftarRekening) {
            if (rekening.getNomorRekening().equals(nomorTujuan)) {
                akunTujuan = rekening;
                break;
            }
        }

        if (akunTujuan == null) {
            System.out.println("Rekening tujuan tidak ditemukan!");
        } else if (akunTujuan == akunAktif) {
            System.out.println("Tidak dapat transfer ke rekening sendiri!");
        } else {
            System.out.print("Masukkan nominal transfer: ");
            double nominal = input.nextDouble();
            input.nextLine();

            System.out.print("Masukkan PIN: ");
            String pinTransfer = input.nextLine();

            akunAktif.transfer(akunTujuan, nominal, pinTransfer);
        }
    }
    break;
```

Penjelasan:

```
if (akunAktif == null) {
    System.out.println("Error: Anda belum membuka rekening!");
```

Jika pengguna memilih menu 8, program akan memeriksa apakah rekening aktif tersedia. Jika belum, program menampilkan pesan “Error: Anda belum membuka rekening

```
} else {
    System.out.print("Masukkan Nomor Rekening Tujuan: ");
    String nomorTujuan = input.nextLine();

    Rekening akunTujuan = null;

    for (Rekening rekening : daftarRekening) {
        if (rekening.getNomorRekening().equals(nomorTujuan)) {
            akunTujuan = rekening;
            break;
        }
    }
```

Jika sudah, pengguna diminta memasukkan nomor rekening tujuan, kemudian program mencari nomor tersebut di dalam daftarRekening

```
  if (akunTujuan == null) {
      System.out.println("Rekening tujuan tidak ditemukan!");
```

Jika rekening tujuan tidak ditemukan, program menampilkan pesan "Rekening tujuan tidak ditemukan!"

```
} else if (akunTujuan == akunAktif) {
    System.out.println("Tidak dapat transfer ke rekening sendiri!");
```

Jika rekening tujuan sama dengan rekening aktif, program menampilkan pesan "Tidak dapat transfer ke rekening sendiri!".

```
    } else {
        System.out.print("Masukkan nominal transfer: ");
        double nominal = input.nextDouble();
        input.nextLine();

        System.out.print("Masukkan PIN: ");
        String pinTransfer = input.nextLine();

        akunAktif.transfer(akunTujuan, nominal, pinTransfer);
    }
}
break;
```

Jika rekening tujuan valid, program meminta nominal transfer dan PIN, kemudian memanggil method transfer() untuk memproses transaksi.

OUTPUT:

1. Buat 2 Rekening, 001 dan 002

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 001
Masukan Nama Pemilik: naysha
Masukan Saldo Awal: 200000
Masukan PIN(6 digit):123456

Piih Produk:
1.Tabungan Umum
2.Giro Bisnis
Piih Produk: 2
Masukan Batas Overdraft:
500000
Rekening atas nama naysha berhasil dibuat.
Rekening Giro Berhasil dibuat.
```

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 002
Masukan Nama Pemilik: acha
Masukan Saldo Awal: 300000
Masukan PIN(6 digit):130407

Piih Produk:
1.Tabungan Umum
2.Giro Bisnis
Piih Produk: 2
Masukan Batas Overdraft:
500000
Rekening atas nama acha berhasil dibuat.
Rekening Giro Berhasil dibuat.
```

2. Lakukan Transfer ke Rekening Lain

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0.Keluar
Pilih Menu:
8
Masukkan Nomor Rekening Tujuan: 001
Masukkan nominal transfer: 100000
Masukkan PIN: 130407
Tarik tunai Giro Rp100000.0 berhasil. Sisa saldo: Rp200000.0
Setor tunai Rp100000.0 berhasil Saldo saat ini: Rp300000.0
Transfer berhasil!

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0.Keluar
Pilih Menu:
```